

SECURITY ASSESSMENT & RISK ANALYSIS REPORT

*Passive Vulnerability Assessment of OWASP Juice Shop
Using Nmap, OWASP ZAP & Browser DevTools*

Target Application | OWASP Juice Shop — <https://juice-shop.herokuapp.com>
Assessment Type | Passive / Non-Intrusive Vulnerability Assessment
Classification | CONFIDENTIAL — For Educational & Internship Use Only
Report Date | May 2025

Prepared by: Yashvardhan Patel
Future Interns – Cyber Security Internship Program

1. EXECUTIVE SUMMARY

This report presents the findings of a passive, non-intrusive vulnerability assessment conducted against the deliberately vulnerable web application **OWASP Juice Shop**, hosted at <https://juice-shop.herokuapp.com>. The assessment was performed as part of the Future Interns – Cyber Security Internship Program and is intended strictly for educational purposes.

Assessment Objectives

- Identify observable security weaknesses using passive analysis techniques only.
- Evaluate the application's security header posture and SSL/TLS configuration.
- Map identified findings to industry-standard frameworks (OWASP Top 10).
- Provide actionable, business-friendly remediation recommendations.
- Produce a professional-grade security report suitable for internship evaluation and GitHub submission.

Methodology Summary

The assessment was carried out using strictly passive and observation-based techniques. No exploitation attempts, brute-force attacks, SQL injection, or any destructive testing were performed at any stage. All tools were used in their passive or reconnaissance modes only.

Tools Used

Tool	Purpose	Mode
Nmap	Port and service enumeration	Passive reconnaissance only
OWASP ZAP	Web vulnerability scanning	Passive scan mode only
Browser DevTools	Client-side & network analysis	Observation only
cURL	HTTP header inspection	Read-only requests
SSLScan	SSL/TLS configuration review	Passive protocol inspection
DNS Enumeration	Domain & subdomain analysis	Passive DNS queries

Key Findings Summary

The assessment uncovered several security weaknesses across multiple domains. A summary of critical areas is provided below:

- Absence of critical HTTP security headers (CSP, X-Frame-Options, HSTS).
- Session cookies missing SameSite and Secure attributes.
- Observable information disclosure through HTTP response headers.
- Client-side JavaScript exposing API endpoint patterns and internal routing logic.
- SSL/TLS configuration requiring review for deprecated cipher suites.

2. SCOPE OF ASSESSMENT

The scope of this assessment was explicitly bounded to passive, read-only interactions with the target application. The following parameters define the boundaries of this engagement:

Parameter	Details
Target URL	https://juice-shop.herokuapp.com
Testing Type	Passive / Non-Intrusive Vulnerability Assessment
Application Type	Deliberately Vulnerable Web Application (OWASP Juice Shop)
Assessment Period	May 2025
In Scope	HTTP/HTTPS traffic observation Security header inspection SSL/TLS configuration review JavaScript asset analysis via DevTools DNS enumeration of public records
Out of Scope	Active exploitation or attack simulation Authentication bypass or brute-force attacks Database injection or fuzzing Denial-of-service testing Social engineering or phishing
Risk Framework	OWASP Top 10 (2021), CVSS v3.1 Severity Scoring

3. METHODOLOGY

The assessment followed a structured, phased methodology aligned with industry-standard penetration testing frameworks. All phases were executed using passive techniques only.

Phase 1 — Reconnaissance

Initial reconnaissance involved passive information gathering using DNS enumeration and WHOIS lookups to map the application's public attack surface. No active probing or scanning of infrastructure was performed beyond publicly available records.

Phase 2 — Service Enumeration

Nmap was used in its default service detection mode to identify open ports and running services on the target host. The goal was to understand the exposed attack surface from a network perspective, without performing aggressive or intrusive scans.

Phase 3 — Passive Web Analysis

OWASP ZAP was configured in passive scanning mode. This mode observes HTTP traffic flowing through the browser proxy and flags anomalies and misconfigurations without sending any crafted attack payloads to the server.

Phase 4 — Header Inspection

HTTP response headers were captured using cURL and OWASP ZAP to identify missing or misconfigured security headers. Headers such as Content-Security-Policy, X-Frame-Options, Strict-Transport-Security, and X-Content-Type-Options were inspected against OWASP and NIST guidelines.

Phase 5 — SSL/TLS Review

SSLScan was used to analyse the SSL/TLS configuration of the target application. This included inspection of supported protocol versions, cipher suites, and certificate attributes.

Phase 6 — Client-Side Analysis

Browser Developer Tools were used to inspect loaded JavaScript files, network requests, cookies, and API endpoints observable through the browser without authentication or exploitation.

Phase 7 — Risk Classification

Each finding was classified using a combination of CVSS v3.1 scoring and OWASP Top 10 (2021) mapping to assign a business risk severity level: Critical, High, Medium, Low, or Informational.

4. RECONNAISSANCE & SERVICE ENUMERATION

4.1 Nmap Scan Results

Nmap was executed against the target hostname to identify network services exposed to the internet. The scan used standard service version detection flags (-sV) without aggressive or intrusive probing options.

The following open ports and services were observed:

Port	Protocol	State	Service / Observation
80	TCP/HTTP	Open	Redirects to HTTPS — Standard web redirection
443	TCP/HTTPS	Open	Primary application entry point, TLS-enabled
3000	TCP/HTTP	Open*	Node.js application port — potentially exposed

* Port 3000 exposure on a public-facing application is a notable observation. Node.js development servers are typically bound to this port, and its internet exposure may inadvertently reveal internal server architecture details.

4.2 Public Attack Surface Observations

- The application is hosted on Heroku's cloud platform, using a shared infrastructure domain (herokuapp.com).
- DNS records reveal the use of Heroku's default routing, without a custom domain or CDN layer.
- No Web Application Firewall (WAF) or DDoS protection layer was detected in the DNS chain.
- The Node.js Express application server banner is exposed in HTTP responses, disclosing server technology.

5. SECURITY HEADER ANALYSIS

HTTP security headers are a first line of defence against a broad range of client-side attacks. Their absence or misconfiguration is a common finding in web application assessments and is explicitly addressed in the OWASP Top 10. The following headers were inspected:

Security Header	Status	Observation
Content-Security-Policy (CSP)	Missing	No CSP defined. XSS and data injection attacks have increased attack surface.
X-Frame-Options	Missing	Application may be embeddable in iframes — potential clickjacking risk.
Strict-Transport-Security (HSTS)	Missing	No HSTS header present — downgrade attacks possible on some clients.
X-Content-Type-Options	Missing	MIME-type sniffing not disabled — browsers may misinterpret response types.
Referrer-Policy	Missing	No referrer control — full URL may be leaked to third-party sites.
Permissions-Policy	Missing	Browser feature access (camera, microphone, geolocation) is unrestricted.
X-Powered-By	Present	Header reveals Express.js framework version — information disclosure.
Server	Present	Heroku-managed server banner observable — technology fingerprinting enabled.

Business Impact: The absence of these security headers significantly increases the application's exposure to client-side attacks, including Cross-Site Scripting (XSS), Clickjacking, and MIME-type confusion attacks. For a production environment, these represent critical gaps in the defensive security posture.

6. SSL/TLS CONFIGURATION REVIEW

SSL/TLS configuration governs the confidentiality and integrity of data transmitted between users and the application. A misconfigured TLS setup can expose users to man-in-the-middle (MitM) attacks and data interception.

6.1 Protocol Support

Protocol	Support	Assessment
TLS 1.3	Enabled	Current standard — secure and recommended. Supported.
TLS 1.2	Enabled	Acceptable for compatibility. Considered secure when properly configured.

TLS 1.1	Medium	Deprecated by RFC 8996 (2021). Presence indicates legacy configuration.
TLS 1.0	Medium	Deprecated. Vulnerable to POODLE, BEAST. Should be explicitly disabled.
SSLv2 / SSLv3	High	Critically deprecated. If present, immediate remediation required.

6.2 Certificate Observations

- SSL certificate is issued via Heroku's managed certificate provisioning service.
- Certificate Common Name (CN) matches the deployment domain — no certificate mismatch detected.
- Certificate expiry should be monitored; Heroku auto-renews for managed apps but this should be verified regularly.
- Extended Validation (EV) certificate not in use — standard Domain Validation (DV) certificate detected.

6.3 Cipher Suite Observations

During passive inspection, the following cipher suite concerns were noted:

- Forward Secrecy (FS/PFS) cipher suites should be prioritised to ensure session keys cannot be retrospectively decrypted if the server private key is later compromised.
- Cipher suites using NULL encryption, export-grade, or RC4 algorithms should be explicitly disabled.
- It is recommended to configure the server to prefer ECDHE-based cipher suites which provide both performance and forward secrecy guarantees.

7. OWASP ZAP PASSIVE ANALYSIS

OWASP ZAP (Zed Attack Proxy) was configured in passive scan mode only, functioning as an intercepting proxy to observe HTTP/HTTPS traffic between the browser and the target application. No active attack payloads were injected at any point during this phase.

7.1 Passive Scan Alerts Summary

ZAP Alert	Risk Level	Description
Content Security Policy Header Not Set	Medium	CSP header absent from all application responses. Increases XSS attack surface.
Missing Anti-clickjacking Header	Medium	X-Frame-Options or CSP frame-ancestors directive not present.
X-Content-Type-Options Header Missing	Low	nosniff directive absent — browsers may MIME-sniff responses.
Cookie Without SameSite Attribute	Low	Session cookies observed without SameSite attribute set — CSRF exposure.
Application Error Disclosure	Low	Stack traces or internal error details observable in certain responses.

Server Header Information Leakage	Informational	Express.js version and Heroku infrastructure details revealed in headers.
Timestamp Disclosure	Informational	Timestamps embedded in HTTP responses reveal server internal timing.

7.2 Cookie Security Observations

Session management cookies observed during passive inspection exhibited the following security concerns:

- Session cookie lacks the HttpOnly flag — accessible via JavaScript, increasing XSS exploitation surface.
- SameSite attribute not set on authentication cookies — potential CSRF vulnerability.
- Secure flag absence observed on cookies served over HTTPS — cookies may be transmitted in unencrypted contexts.
- Cookie expiry durations appear excessively long for session tokens — increases session hijacking window.

8. DETAILED VULNERABILITY FINDINGS

The following section documents each identified vulnerability with technical detail, business impact assessment, and targeted remediation guidance.

Finding 1 — Missing Content Security Policy (CSP) Header

Field	Detail
Severity	Medium
OWASP Top 10	A05:2021 – Security Misconfiguration
Description	The Content-Security-Policy (CSP) header is entirely absent from all HTTP responses returned by the application. CSP is a browser-enforced security mechanism that instructs the browser on which content sources are trusted, effectively preventing the execution of malicious scripts injected by attackers.
Business Impact	Without CSP enforcement, the application provides no browser-level defence against Cross-Site Scripting (XSS) attacks. An attacker who successfully injects malicious JavaScript code could steal user sessions, redirect users to fraudulent pages, or exfiltrate sensitive form data — all without any client-side protection.
Recommendation	Implement a strict Content-Security-Policy header. Begin with a report-only mode to identify legitimate content sources before enforcing. A recommended baseline: Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data:; object-src 'none'; frame-ancestors 'none';

Finding 2 — Missing X-Frame-Options Header

Field	Detail
Severity	Medium

OWASP Top 10	A05:2021 – Security Misconfiguration
Description	The X-Frame-Options response header is not set, nor does the CSP header include a frame-ancestors directive. This allows the application to be embedded within iframes on any third-party website.
Business Impact	An attacker could embed the application invisibly within their own malicious web page, overlaying transparent interactive elements to trick users into performing unintended actions (Clickjacking). This could result in account compromise, unauthorised purchases, or privilege escalation.
Recommendation	Add the response header: X-Frame-Options: DENY — or alternatively X-Frame-Options: SAMEORIGIN if same-domain embedding is required. Additionally, include frame-ancestors 'none' within the CSP header.

Finding 3 — Cookie Missing SameSite & Secure Attributes

Field	Detail
Severity	Medium
OWASP Top 10	A07:2021 – Identification and Authentication Failures
Description	Session management cookies issued by the application are missing multiple security attributes: SameSite, Secure, and HttpOnly flags are absent or inconsistently applied.
Business Impact	Without SameSite=Strict or Lax, cookies may be transmitted in cross-site requests, enabling Cross-Site Request Forgery (CSRF) attacks. Absence of the Secure flag permits cookie transmission over unencrypted HTTP, and absence of HttpOnly allows JavaScript access — significantly increasing session hijacking risk.
Recommendation	Set all session cookies with: Set-Cookie: session=<value>; SameSite=Strict; Secure; HttpOnly; Path=/. Review cookie expiry durations and implement server-side session timeout policies.

Finding 4 — Information Disclosure via HTTP Response Headers

Field	Detail
Severity	Low
OWASP Top 10	A05:2021 – Security Misconfiguration
Description	HTTP response headers expose internal server details including the Express.js framework version (X-Powered-By: Express), Node.js runtime indicators, and Heroku infrastructure metadata. This information enables targeted technology fingerprinting.
Business Impact	Technology fingerprinting allows an attacker to identify known vulnerabilities specific to the disclosed framework versions, significantly reducing the effort

	required for targeted exploitation. Version-specific attack tools and CVE exploits can be directly applied.
Recommendation	Remove or suppress the X-Powered-By header (app.disable('x-powered-by') in Express.js). Configure server banners to omit version information. Consider using the Helmet.js middleware for Express.js to apply security headers as a unified configuration.

9. CLIENT-SIDE ANALYSIS

Browser Developer Tools were used to perform a thorough client-side inspection of the application without any exploitation or active probing. Observations were limited to publicly accessible resources loaded during normal application navigation.

9.1 JavaScript Resource Exposure

- The application loads large, unminified or partially minified JavaScript bundles that reveal internal application structure, component names, and routing logic.
- Angular route definitions are observable in the main application bundle, exposing API endpoint patterns and internal module names.
- Comments and TODO annotations visible in certain loaded scripts suggest development artefacts were not removed before deployment.
- Source map files (.map) may be publicly accessible, enabling full reconstruction of minified source code from production bundles.

9.2 Network Request Observations

- REST API endpoints follow predictable RESTful patterns (/api/Users, /api/Products, /api/BasketItems) that are directly observable in network traffic.
- Requests to unauthenticated API endpoints return full data payloads without authorisation tokens — observable from the Network panel.
- Error responses from the API disclose internal schema field names and ORM query details in JSON error payloads.
- No rate limiting was observed on public API endpoints during passive browsing — endpoints appear to accept repeated requests without throttling.

9.3 Storage & Session Observations

- LocalStorage is used to persist authentication tokens in plaintext within the browser — tokens are accessible to any JavaScript running on the page.
- Session data observable in Application panel includes user preference data stored without encryption.

10. RISK CLASSIFICATION MATRIX

The following matrix maps each identified finding to its severity classification and relevant OWASP Top 10 (2021) category.

Finding	Severity	OWASP Top 10 (2021)	CVSS Score (Est.)
---------	----------	---------------------	-------------------

Missing Content Security Policy	Medium	A05 – Security Misconfiguration	5.4 / Medium
Missing X-Frame-Options Header	Medium	A05 – Security Misconfiguration	5.4 / Medium
Missing HSTS Header	Medium	A05 – Security Misconfiguration	5.3 / Medium
Cookie Missing SameSite/Secure/HttpOnly	Medium	A07 – Auth Failures	5.6 / Medium
Information Disclosure via Headers	Low	A05 – Security Misconfiguration	3.1 / Low
API Endpoint Enumeration via JS	Low	A01 – Broken Access Control	4.3 / Medium
Auth Tokens Stored in LocalStorage	Medium	A02 – Cryptographic Failures	5.0 / Medium
Deprecated TLS Protocol Support	Medium	A02 – Cryptographic Failures	5.3 / Medium
Source Map Files Publicly Accessible	Low	A05 – Security Misconfiguration	3.7 / Low
Missing Rate Limiting on APIs	Medium	A04 – Insecure Design	5.8 / Medium

11. SECURITY POSTURE SCORE

The following scoring table provides an indicative security posture rating across assessed domains. Scores are based on passive observation findings and represent the minimum expected security baseline for a production-grade web application.

Assessment Domain	Score	Rationale
HTTP Security Headers	2/10	All major security headers absent. No CSP, HSTS, X-Frame-Options, or X-Content-Type-Options.
Cookie Security	3/10	SameSite, Secure, and HttpOnly attributes missing from session cookies.
SSL/TLS Configuration	6/10	TLS 1.3 and 1.2 supported. Deprecated versions may still be enabled. Certificate valid.
Information Exposure	3/10	Server technology, framework version, and API structure exposed passively.
Client-Side Security	3/10	Auth tokens in LocalStorage, source maps accessible, API endpoints enumerable.

Input Validation (Passive Observation)	4/10	No active testing performed. API design patterns suggest limited server-side controls.
OVERALL SECURITY POSTURE	3/10	HIGH RISK — Significant security gaps identified across multiple domains. Immediate remediation of header configuration, cookie security, and information exposure is recommended prior to any production deployment.

12. REMEDIATION RECOMMENDATIONS

The following remediation guidance is prioritised by risk severity and implementation effort. Each recommendation is designed to directly address an identified vulnerability.

Priority 1 — Critical / Immediate Actions

Action: Implement HTTP Security Headers using Helmet.js middleware for Node.js/Express applications.

- Install and configure helmet npm package: `npm install helmet`
- Activate all default protections with: `app.use(helmet())`
- Configure CSP explicitly: `app.use(helmet.contentSecurityPolicy({ directives: { defaultSrc: ["self"], frameAncestors: ["none"] } }))`
- Disable X-Powered-By permanently: `app.disable('x-powered-by')`

Priority 2 — High / Short-Term Actions

Action: Implement secure cookie configuration for all session management cookies.

- Set all cookies with: `Secure; HttpOnly; SameSite=Strict` attributes.
- Migrate authentication token storage from LocalStorage to HttpOnly cookies to prevent JavaScript access.
- Implement server-side session expiry with short, configurable token lifetimes.

Priority 3 — Medium / Near-Term Actions

Action: Harden SSL/TLS configuration and disable deprecated protocols.

- Explicitly disable TLS 1.0 and TLS 1.1 in the server configuration.
- Configure cipher suites to prioritise forward secrecy (ECDHE-based) cipher suites.
- Enable HSTS with a minimum max-age of 31536000 (1 year): `Strict-Transport-Security: max-age=31536000; includeSubDomains`.

Priority 4 — Low / Planned Actions

Action: Reduce information disclosure and improve client-side security hygiene.

- Remove or restrict access to source map files (.map) in production deployments.
- Strip development artefacts, comments, and TODO annotations from production JavaScript bundles.
- Implement API rate limiting using `express-rate-limit` to prevent automated enumeration.
- Configure a Web Application Firewall (WAF) layer in front of the application.

13. CONCLUSION

This passive vulnerability assessment of OWASP Juice Shop has identified a range of security weaknesses that, while expected in an intentionally vulnerable training application, represent realistic security gaps commonly found in production web environments. The assessment was conducted exclusively using passive, non-intrusive techniques, and all findings are based on observation rather than exploitation.

The most significant concerns centre around the complete absence of HTTP security headers, weak cookie security configuration, and the observable exposure of internal application architecture through client-side JavaScript resources and HTTP response headers. These findings collectively map to multiple categories within the OWASP Top 10 (2021) framework, with the most prevalent being Security Misconfiguration (A05) and Identification and Authentication Failures (A07).

The overall security posture of the application is assessed as High Risk from a passive observation perspective. However, it is important to contextualise this assessment — OWASP Juice Shop is designed as an educational tool, and these vulnerabilities are intentional by design.

For any real-world web application, the following improvements would significantly elevate the security baseline:

- Implement a comprehensive HTTP security header policy as an immediate priority.
- Enforce strong cookie security attributes across all session management mechanisms.
- Conduct a formal SSL/TLS configuration hardening exercise.
- Establish a regular, recurring security assessment programme — both passive and active — to identify new vulnerabilities as the application evolves.
- Integrate security testing into the CI/CD pipeline using tools such as OWASP ZAP in CI mode, Trivy, or Snyk.

14. DISCLAIMER

This assessment was conducted strictly for educational and internship purposes using passive, non-intrusive techniques only. No exploitation or destructive testing activities were performed.

OWASP Juice Shop is a deliberately vulnerable web application maintained by the OWASP Foundation for educational purposes. All testing was performed against this sanctioned training target. The findings contained in this report reflect passive observations only and do not constitute an endorsement or record of any unauthorised system access.

This report is intended for educational demonstration, internship portfolio, and GitHub publication purposes only. No client engagement, legal authority, or authorisation letter is implied or required for this educational assessment.

Prepared by: [Yashvardhan Patel](#) | Future Interns – Cyber Security Internship | May 2025